

A Lightweight Architecture for Hardware-Based Security in the Emerging Era of Systems of Systems

NICO MEXIS, University of Passau, Germany

NIKOLAOS ATHANASIOS ANAGNOSTOPOULOS, Technical University of Darmstadt, Germany and University of Passau, Germany

SHUAI CHEN, Southeast University, China and Rock-Solid Security Lab., China

JAN BAMBACH, Technical University of Darmstadt, Germany

TOLGA ARUL, Technical University of Darmstadt, Germany and University of Passau, Germany

STEFAN KATZENBEISSER, University of Passau, Germany

In recent years, a new generation of the Internet of Things (IoT 2.0) is emerging, based on artificial intelligence, the blockchain technology, machine learning, and the constant consolidation of pre-existing systems and subsystems into larger systems. In this work, we construct and examine a proof-of-concept prototype of such a system of systems, which consists of heterogeneous commercial off-the-shelf components, and utilises diverse communication protocols. We recognise the inherent need for lightweight security in this context, and address it by employing a low-cost state-of-the-art security solution. Our solution is based on a novel hardware and software co-engineering paradigm, utilising well-known software-based cryptographic algorithms, in order to maximise the security potential of the hardware security primitive (a Physical Unclonable Function) that is used as a security anchor. The performance of the proposed security solution is evaluated, proving its suitability even for real-time applications. Additionally, the Dolev-Yao attacker model is considered in order to assess the resilience of our solution towards attacks against the confidentiality, integrity, and availability of the examined system of systems. In this way, it is confirmed that the proposed solution is able to address the emerging security challenges of the oncoming era of systems of systems.

CCS Concepts: • **Computer systems organization** → *Sensor networks*; **Heterogeneous (hybrid) systems**; • **Networks** → *Network security*; *Cyber-physical networks*; • **Security and privacy** → **Hardware security implementation**; *Network security*; • **Hardware** → **Analysis and design of emerging devices and systems**;

Nico Maxis, Nikolaos Athanasios Anagnostopoulos, Shuai Chen, and Tolga Arul contributed equally to this work.

This work has been funded by the German Research Foundation—Deutsche Forschungsgemeinschaft (DFG), under Projects “PUFMem: Intrinsic Physical Unclonable Functions from Emerging Non-Volatile Memories” (AR 1387/1-1 | KA 2308/4-1) and “NANOSEC: Tamper-Evident PUFs Based on Nanostructures for Secure and Robust Hardware Security Primitives” (HE 6457/4-1 | KA 2308/3-1) of the Priority Program “Nano Security: From Nano-Electronics to Secure Systems” (SPP 2253) (<https://spp-nanosecurity.uni-stuttgart.de/>).

Authors’ addresses: N. Maxis and S. Katzenbeisser, University of Passau, Faculty of Computer Science and Mathematics, Innstraße 43, Passau, Bayern 94032, Germany; emails: mexis01@ads.uni-passau.de, info@femtopedia.de, stefan.katzenbeisser@uni-passau.de; N. A. Anagnostopoulos and T. Arul, Technical University of Darmstadt, Department of Computer Science, Hochschulstraße 10, Darmstadt, Hessen 64289, Germany and University of Passau, Faculty of Computer Science and Mathematics, Innstraße 43, Passau, Bayern, 94032, Germany; emails: {anagnostopoulos, arul}@seceng.informatik.tu-darmstadt.de, {nikolaos.anagnostopoulos, tolga.arul}@uni-passau.de; S. Chen, Southeast University, School of Microelectronics, Sipailou Road 2, Nanjing 210000, China and Rock-Solid Security Lab., Jinjihu Road 1335, Suzhou 215000, China; email: chenshuai_ic@seu.edu.cn, chenshuai_ic@163.com; J. Bambach, Technical University of Darmstadt, Department of Computer Science, Hochschulstraße 10, Darmstadt, Hessen 64289, Germany; email: jan.bambach@stud.tu-darmstadt.de.

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

© 2021 Association for Computing Machinery.

1550-4832/2021/06-ART43 \$15.00

<https://doi.org/10.1145/3458824>

Additional Key Words and Phrases: System of systems, Internet of Things, hardware and software security co-engineering, IoT 2.0

ACM Reference format:

Nico Mexis, Nikolaos Athanasios Anagnostopoulos, Shuai Chen, Jan Bambach, Tolga Arul, and Stefan Katzenbeisser. 2021. A Lightweight Architecture for Hardware-Based Security in the Emerging Era of Systems of Systems. *J. Emerg. Technol. Comput. Syst.* 17, 3, Article 43 (June 2021), 25 pages.
<https://doi.org/10.1145/3458824>

1 INTRODUCTION AND BACKGROUND INFORMATION

The **Internet of Things (IoT)** is a concept that has gained ever-increasing popularity over recent years. The IoT describes an interconnected network on which devices communicate with each other, in order to process sensor data and take actions, accordingly, without human intervention or action. For this reason, the notion of the IoT is highly related to applications and scientific fields that involve sensors and actuators. While, initially, the IoT was rather limited to quite restrained devices and environments, and found application in sensor networks and in areas such as agriculture, the nature of the IoT has increasingly become almost ubiquitous, with its utilisation ranging from autonomous transportation and smart energy grids to biomedical and space applications. Additionally, by its nature, the IoT, which bases its successful operation on the collection of sensor data in order to determine the right course of actions, is highly related to the rapidly developing scientific fields of machine learning, big data collection, and analysis, as well as to the areas of artificial intelligence and blockchain technologies.

In this context, a new generation of the IoT (IoT 2.0) has started to emerge, which takes advantage of the recent developments in the fields of big data, machine learning, and artificial intelligence, in order to provide further capabilities and improved functionality to the different systems that comprise the IoT. Such systems are not only found in commercial applications, but also in the industry, especially within the framework of the Industry 4.0 technology, forming what is known as the **Industrial IoT (IIoT)**. Hence, not only has the IoT become almost ubiquitous and is widely adopted in a broad range of different sectors and applications, but it has also led to the emergence of new synergies within pre-existing as well as novel systems. In particular, IoT systems (which may consist of a large number of subsystems), such as smart houses, autonomous vehicles and their relevant infrastructure, agricultural IoT segments, and even space IoT segments, have now started to communicate with each other, forming larger systems of systems, in order to better address the particular demands and requirements of specific applications and use case scenarios. Such an evident example, already introduced, up to some degree, in the literature [3] would be the case of a fire in a rural area, where the aforementioned IoT segments should act as an ad hoc **System of Systems (SoS)**, in order to detect the fire, notify the relevant authorities and, at the same time, take adequate action to ensure the safety of humans that are potentially present in the affected area, including nearby residences and vehicles, while also taking any action possible to contain or even put out the fire.

Nevertheless, such ad hoc systems of systems are not limited to emergency use cases, but rather form the core of the existing IoT and of the oncoming IoT 2.0 generation of devices, networks, and general infrastructure. Particular examples of such systems of systems include systems composed of weather prediction sensor networks and agricultural IoT segments, autonomous cars and road infrastructure, or even autonomous vehicles and (space and terrestrial) global positioning systems. In all these cases, each individual component is a system of its own, or even a system of systems of its own. For example, an autonomous vehicle incorporates a braking system which bases its actions

on information coming from its own sensor systems (radar, lidar, video camera acquisition, etc.) and utilises its own actuator systems (brakes). In the same way, the road infrastructure is composed of on-road sensor systems, off-road computation systems, as well as on-road imaging systems (e.g., configurable LED indicator boards for traffic signs to be displayed on them). Additionally, when these two systems of systems are combined to form an overall system for safe and secure autonomous transportation, the systems utilised for their mutual communication need to also be considered.

Such systems of systems can also be easily used for entertainment and leisure activities, especially in the context of IoT 2.0. For example, the simple intention of a human to watch on the **television (TV)** a particular movie, or football game, while eating, when she/he reaches home at night after work, would require the interaction of at least three different systems of systems in the context of the IoT 2.0 (i.e., the road and general environmental infrastructure, the smart (potentially autonomous) vehicle, and the smart home of the relevant person). Additional systems of systems, such as satellite IoT segments, may also be involved, in order to determine the position of the person involved in relevance to time, as well as the environmental conditions (especially with regard to the road). This information would then be transferred not only to the relevant smart vehicle, but also to the smart home involved, which may need to record (at least parts of) the movie or the football match, if it is being transmitted in real time, or increase, or lower, the speed at which food is being cooked or drinks are being cooled. In order to do so, the subsystems of a smart TV, a smart kitchen stove, and a smart fridge may need to form a smart home system. Naturally, each of these subsystems is composed of other subsystems of its own. Thus, it becomes apparent that the IoT is not only evolving to a new generation (IoT 2.0) that takes advantage of machine learning and artificial intelligence, but is also progressively introducing a new era of system interaction and architecture, the *emerging era of systems of systems*.

Undoubtedly, security is a major concern for the IoT in the context of systems of systems, as such systems may have a rather ad hoc nature and be extremely volatile, as already discussed. However, recent efforts to address the security of the IoT focus rather on its specifics (i.e., controller devices, networks, sensors, and actuators) and rather fail to propose a holistic approach that genuinely considers the overall system, especially in the context of a system composed of other systems. Most of the existing security approaches are rather aimed at securing the individual (sub)systems or their components, without truly addressing the overall structure and architecture of a highly heterogeneous SoS. For this purpose, our work concerns the implementation of a proof-of-concept prototype of an SoS that is heterogeneous in terms of both the devices that comprise it and the network protocols that it utilises. This implementation can, of course, be further extended in the future to include even more heterogeneous devices and network protocols. Nevertheless, it does provide a testbed suitable for examining potential security solutions for the IoT in the emerging era of systems of systems, due to its highly heterogeneous nature and its resemblance to a realistic (albeit rather limited) SoS. Further, we utilise this proof-of-concept implementation in order to apply and examine our proposed security solution for currently employed and next-generation IoT devices and networks in the context of systems of systems.

1.1 A Brief Overview of the Security Solution Presented in This Article

In this work, we propose a security solution that is based on hardware and software co-engineering, in order to address the potential security weaknesses that hardware and software may have on their own. In particular, we combine well-known and well-tested cryptographic algorithms, i.e., HMAC-SHA-256, and AES-128-CBC encryption with PKCS#7 padding, with a well-known hardware security primitive, namely, a **Physical Unclonable Function (PUF)**.

More specifically, the time information of the messages sent in our network protocol is authenticated using a nonce and a **Hash-based Message Authentication Code (HMAC)** (in particular, the HMAC-SHA-256), through a local broker node. Then, the timestamps of data messages, as well as the data sent, are encrypted using the AES-128-CBC encryption algorithm with PKCS#7 padding. All the keys being used by the software schemes are based on the responses of a **Dynamic Random Access Memory (DRAM)** decay-based PUF. Therefore, the security anchor of the proposed security scheme is a hardware primitive, namely, a PUF implemented using the decay characteristics of the DRAM of a Raspberry Pi 3B+ development board.

In general, it has been shown in recent works that PUFs can constitute adequate security mechanisms for IoT devices [2, 4, 15]. PUFs utilise minor manufacturing variations, which do not affect the correct operation of the circuits of a component, but which lead to identifiable characteristics of such components. Each PUF can act as a function embedded in hardware, as, for each particular input, being referred to as a *challenge*, it should provide a particular output, being referred to as a *response*.

However, in practice, PUF responses tend to be noisy and, therefore, a bit selection/correction mechanism needs to be applied to each one of them, in order to both stabilise that response and, at the same time, also transform it into a cryptographic token, such as a key. Although this bit selection/correction mechanism is most often based upon the use of a (reverse) fuzzy extractor scheme [9, 10, 35], it can also be a simple helper data scheme that selects the most stable bits of each response, in order to produce a key. In this work, we indeed utilise a simple scheme that selects stable response bits in order to construct a cryptographic key, as this mechanism has proven to be both extremely simple to implement as well as immensely efficient, for DRAM decay-based PUFs [31].

In general, the **Challenge-Response Pair(s) (CRP(s))** of a PUF can be used to implement a large number of security solutions [3, 14, 16, 17, 22, 24, 32], such as identification, authentication, attestation, anti-counterfeiting, secure boot, secure communication, and secure key agreement protocols. Additionally, PUFs can also be used as a source of randomness [6, 16, 17] and for the realisation of (true or pseudo) random number generators. Moreover, based on the amount of their CRPs, PUFs can be classified into *weak PUFs*, which allow for only a very limited number of CRPs, and *strong PUFs*, which allow for a large number of CRPs, such that their complete characterisation within a limited time frame is not possible [18, 38]. Furthermore, memory-based PUFs are most often based upon inherent system components and, therefore, have the advantage of being considered *intrinsic*, as they do not require the addition of hardware for their construction or operation.

In this work, we utilise a weak memory-based PUF as the security anchor of the scheme. This choice significantly increases the security of the scheme, as potential attackers need to gain physical access to the (hardware) device itself, or at least privileged software access to it, in order to be able to compromise present and future keys. Additionally, weak memory-based PUFs have been proven to be lightweight, cost-efficient, scalable, flexible, and, in general, very practical to use [4].

Even though PUFs have proven to be an adequate security mechanism for both currently deployed and next-generation IoT networks and devices, a security solution based only on a hardware primitive does not always fully address the relevant security needs in the context of systems of systems. In particular, the emerging era of systems of systems calls for a generic security solution that can be applied to different use cases where highly heterogeneous devices and networks are combined in an ad hoc manner [20]. Thus, the proposed security solution needs to be lightweight and cost-efficient (due to the cost-efficient nature of the IoT, which is based on economies of scale), as well as flexible and scalable (due to the ad hoc and diverse nature of systems of systems), in order to be efficient and practical (and, thus, provide adequate security in a feasible manner).

This need can only be addressed through a novel security architecture paradigm that is based on hardware and software co-engineering, in order to take advantage not only of the benefits of using a state-of-the-art hardware security primitive, but also of the leading cryptographic algorithms, and the most efficient data exchange protocols. This novel security concept can also provide more enhanced security than individual software and hardware security mechanisms, and potentially mitigate the shortcomings that such security mechanisms have on their own, while still being lightweight and, therefore, relevant for adoption in the varying applications of the IoT in the emerging era of systems of systems.

Our work proposes exactly such a security scheme, introducing a novel architecture for securing current and future IoT devices, networks, and systems, in the context of systems of systems. It combines a lightweight state-of-the-art hardware security primitive (a DRAM PUF) with lightweight state-of-the-art authentication (HMAC-SHA-256) and encryption (AES-128-CBC) software algorithms, while it utilises the **Message Queuing Telemetry Transport (MQTT)** protocol for efficient data exchange. We need to note that, in order for the solution to be considered as practical, the chosen security primitives, cryptographic algorithms, and data exchange protocols not only need to be considered as efficient and effective, but also to have been confirmed as secure, after sufficient testing. Thus, our solution is based upon proven state-of-the-art security and communication components and not on features and components that are still considered to be in an experimental phase. Nevertheless, of course, in the future, the usage of different features and components may prove to be more beneficial. However, we believe that the concept of hardware and software co-engineering that our work proposes will still inevitably form the standard for security in the field of systems of systems.

Moreover, it needs to be noted that in order to keep our solution cost-efficient and practical, we have utilised **Commercial Off-The-Shelf (COTS)** devices and components for our system-of-systems implementation, upon which our security solution is tested. It is very important to note the role of rather low-end hardware, both in the implementation of a realistic system-of-systems proof-of-concept prototype and in the testing of a practical security solution for such an SoS, as a system of systems will inevitably include some low-end devices, which will be rather resource-constrained. In the same context, we have also considered, in our work, that often some segments of a system of systems may have particular requirements, for example, regarding real-time communication. To this end, we have tried to make sure that our proof-of-concept implementation of a system of systems allows for the evaluation of such requirements and that our relevant security solution takes such issues into some consideration. However, it has, of course, proven impossible to consider all potential requirements for the different use cases of systems of systems in this work, and, to this extent, we observe that other potential requirements, such as safety-related issues, fall at present outside the scope of the work presented in this article, but ought to be examined in other future works.

1.2 Contributions and Scope of This Article

This article makes the following contributions:

- It addresses the need for security in heterogeneous IoT networks incorporating a variety of systems of systems, focusing on the security of the overall system, rather than securing *only* individual components or (sub)systems.
- This work proposes a lightweight security scheme that is based on both hardware and software security primitives. This essentially introduces a novel security architecture paradigm for the IoT that is based on a hardware and software co-engineering concept, while at the same time being lightweight and cost-efficient.

- This article presents a real implementation of a secure system of systems that utilises heterogeneous COTS devices, which is secured using state-of-the-art hardware and software security primitives, therefore proving the feasibility and novelty of the proposed solution. A brief overview of the relevant literature proves that previous works consider the security of such a system (the overall system of systems) only in a very limited manner, which significantly limits the practical applicability of such works.
- In this article, the proposed security scheme is assessed as potentially able to handle real-time communications, as the overall time required by the proposed security protocol to handle round-trip communication *and* network packet processing rarely exceeds 30 ms. This result indicates that the proposed protocol can find application in use cases requiring real-time communication, such as in-vehicle communications and other time-critical applications.
- The presented protocol is assessed against a number of attack vectors, based on the Dolev-Yao attacker model [11], and the potential attacks are classified as affecting the confidentiality, the integrity, or the availability of the system. Our work also justifies why some attacks are not addressed, in order to keep the security solution practical and applicable to the IoT. This makes our work truly relevant for real-world systems, as some attacks can never be fully mitigated, e.g., an attacker may always destroy some (sub)system, causing an unavoidable denial of service.

We also note here that our work considers only the security architecture required for the IoT in the context of systems of systems, and does not address its implications regarding the safety provided by such systems. Nevertheless, the extremely short time required for the proposed security protocol (in almost all cases less than 30 ms, and most often less than 15 ms), allows us to conclude that the proposed solution seems to be applicable even in time-critical use cases, and should not significantly affect the safety provided by such systems. However, our work does not consider safety issues of such systems, or the required safety and security co-engineering of such systems. Therefore, this issue falls outside the scope of this work and should be addressed by future works.

In general, our work aims at being applicable to a broad range of use case scenarios and applications, not being limited to specific safety-critical (and time-critical) applications. For this reason, it cannot be considered as truly applicable to safety-critical (or even time-critical) use cases without further research. To this end, each specific safety-critical (or time-critical) use case needs to be separately examined and evaluated in order to define the extent to which our solution may be applicable to it. For example, our solution appears to be applicable to in-vehicle communication; however, the extent to which this is true depends highly on the particular implementation utilised in such a system of systems, the exact time constraints set by the manufacturer, and the time and safety specifications and requirements imposed by the relevant regulators.

On the whole, we believe that the security solution presented in this work is largely applicable to most, if not to all, of the different IoT use cases and applications in the context of systems of systems, which range from agriculture to autonomous transportation, and from energy grids to space applications. However, it still needs to be assessed if this is truly the case for use cases that inherently impose additional requirements regarding time, safety, and/or other constraints.

1.3 Outline of the Rest of This Work

The rest of this work is organised in the following way. Section 2 presents a brief overview of works relevant to this article. Section 3 describes the experimental setup, with a particular focus on the proof-of-concept prototype of a system of systems that has been implemented, and the proposed security solution, including the relevant security protocol. Subsequently, a brief evaluation of the performance of the proposed solution is presented in Section 4, in order to validate its flexibility,

scalability, and lightweightness. The security of the proposed protocol is discussed in Section 5, under the assumption of a Dolev-Yao attacker model, and with a focus on attacks against the confidentiality, integrity, and availability of the implementation. Finally, Section 6 concludes this work and suggests potential future research.

2 RELATED WORK

In recent years, novel solutions for improving the security of IoT devices have been published continuously. Many of these solutions rely on rather costly methods or functions that are not readily available in many IoT devices. The reason for this is that the adoption of the IoT has historically been dependent on economies of scale. Currently (2020), an average of 127 new IoT devices are connected to the web every second and the total number of IoT devices has been estimated to be between 20 billion and 100 billion [12, 15]. According to predictions,¹ more than 75 billion devices will be connected to the web by 2025. All these devices are already being employed in widely varying applications, ranging from agriculture to autonomous transportation, and from energy grids to space applications. Naturally, these devices, and the relevant IoT networks, vary significantly from each other, in terms of resources and capabilities. Especially due to the widely heterogeneous nature of IoT devices and networks, it becomes apparent that expensive security solutions cannot be considered as feasible or scalable. Therefore, only a lightweight and cost-efficient security solution can be considered as feasible and scalable not only for the next generation of the IoT, but even for the currently deployed one.

Regarding hardware-based security solutions for the IoT, a number of recent publications exist, which, however, fail to fully address the issue of security in the IoT, especially in the emerging era of systems of systems. In particular, PUFs have already been proposed as a solution to improve the security of IoT devices and networks. Here, we provide a brief review of a number of recent works discussing PUFs as a security mechanism for the IoT.

In 2016, Halak et al. [15] published an overview of PUF-based security solutions for modern IoT networks. The security solutions examined by the authors consume few resources and can therefore be used to enhance the security of IoT devices and networks, which are most often resource-constrained. The authors also discuss the reliability of PUF-based security solutions for IoT devices and networks, and the quality of the security provided. However, this work is only an overview of such security solutions and does not include the exact implementation details of a protocol or even a proof-of-concept implementation.

The work of Mukhopadhyay [25], also presented in 2016, provides another overview of PUF-based security solutions for the IoT. The author notes that security issues hamper the adoption of the IoT and discusses a potential security vulnerability of a common IoT setup. He notes that PUFs can act as a low-cost security mechanism for IoT devices, although a number of related issues still persist. The author also proposes a lightweight PUF construction and attacks against it, as well as an authentication protocol and corresponding attacks against it. Finally, the potential of PUF-based protocols for securing IoT devices and networks is discussed. However, in this work, only a strong PUF implementation, which may potentially be very vulnerable to modeling and machine learning attacks [19, 29, 30, 37], is considered.

The work of Wallrabenstein [36] from 2016, notes that IoT devices require cost-efficient security solutions that can be implemented using PUFs. The author presents and evaluates an authentication protocol for IoT devices that utilises both PUFs and elliptic curves. The authentication protocol proposed is an improved version of an existing PUF-based protocol published in the relevant literature. The author also proposes a number of PUF-based algorithms that can be utilised

¹<https://securitytoday.com/Articles/2020/01/13/The-IoT-Rundown-for-2020.aspx?Page=2>.

for device enrollment, authentication, decryption, and digital signature generation, and evaluates their performance on a resource-constrained device. In this way, the suitability of these algorithms for enhancing the security of IoT devices is proven. However, also the solution proposed in this work relies on a strong PUF, which may potentially be very vulnerable to modeling and machine learning attacks [19, 29, 30, 37].

Finally, in 2019, Chen et al. [5] published a paper on intrinsic DRAM PUFs with an implementation on a Raspberry Pi B+. For the implementation, the rpi-open-firmware project of Kristina Brooks² was modified and a separate kernel was written. This kernel opens an interface via the **Universal Asynchronous Receiver-Transmitter (UART)** module, which can be used to send certain parameters to the firmware. By using the Linux mailbox framework, these parameters are sent from the CPU to the GPU, which then utilises the DRAM of the Raspberry Pi to provide a DRAM decay-based PUF according to the parameters received. In our work, we extend the work of Chen et al., so that cryptographic keys can be generated using the relevant DRAM PUF, and include the modified implementation in our SoS design. Chen et al. have also analysed the quality of the proposed DRAM PUF implementation, in terms of its uniqueness, robustness and randomness, proving its ability to act as a secure key storage/generation mechanism. However, the DRAM PUF presented by Chen et al. appears to be dependent on ambient temperature variations. Nevertheless, back in 2018, Anagnostopoulos et al. [4] demonstrated that DRAM decay-based PUFs can be utilised to provide security in a robust way, even though they are temperature-dependent. Therefore, in our work, we utilise the DRAM PUF proposed by Chen et al. in such a way as to provide robust key generation for our security protocol.

Regarding the security of systems of systems, no comprehensive solution seems to have been proposed in the relevant literature. In 2017, Naik and Maral [26] published a rather superficial work about a proof-of-concept implementation of an SoS using the MQTT protocol and how it could be secured using device IDs, session keys, and secret keys. This security solution yields rather similar time performance to the one we propose in our work. However, while the solution proposed by Naik and Maral appears to be resource-efficient, device IDs can be sniffed and subsequently spoofed. The authors try to address this problem, in an effort to prevent replay attacks, by allowing only a single device to be connected for each particular device ID being used. However, this mitigation proposal cannot be considered as optimal, as the attacker could, e.g., wait for a power outage of the legitimate device whose ID should be spoofed, or attempt to disable it, or disrupt its access to the network, in order to launch a successful attack.

In general, the relevant literature dealing with systems of systems security appears to be segmented, which rather indicates that this field is still in its infancy. Although security is a cross-cutting issue, we observe that SoS security has not yet been examined holistically and in depth, as existing works seem to focus either only on high-level security aspects or only on securing individual SoS use cases. High-level aspects of SoS security that have already been studied include models for the specification of security requirements [27], the assessment of security risk and requirements [21], as well as the identification and analysis of attack models [39] for systems of systems. In direct contrast to works broadly examining high-level aspects of SoS security, other works have examined only a single very specific system of systems, and solutions for its particular security requirements and characteristics, such as automotive systems of systems [1, 13], cellular networks [23], smart cities [8], and smart grids [34].

Based on this brief review of the literature relevant to our work, we can conclude that, to the best of our knowledge, none of the existing works has explored a cost-efficient solution for security in systems of systems in the context of the IoT through an actual proof-of-concept

²<https://github.com/christinaa/rpi-open-firmware>.

implementation. Therefore, we do note a relative void in the literature regarding the security of IoT systems in the framework of systems of systems, which our work aims to help investigate and address. Consequently, we believe that our work is the first that presents and examines in depth a comprehensive security solution for IoT systems of systems. In particular, our solution employs a security architecture that is based upon hardware and software co-engineering, in order to address the examined SoS in a holistic manner that considers both hardware and software by design. Furthermore, our proposed security solution is lightweight and cost-efficient as it is based upon COTS hardware components and lightweight software. Finally, in contrast to other security solutions, the proposed approach is also highly scalable and flexible, since it does not significantly affect the performance of the overall SoS and can be easily employed in different use cases and applications.

3 IMPLEMENTATION OF A PROOF-OF-CONCEPT PROTOTYPE OF A SYSTEM OF SYSTEMS

In this section, we will present our implementation of the PUF-secured protocol for data exchange in networks of IoT systems in the SoS era. Our SoS consists of evaluation board computers, such as Raspberry Pis and similar devices. Most of these minicomputers have a Wi-Fi chip, which facilitates communication by allowing the use of predefined functions in their **Application Programming Interface (API)**. Nevertheless, we also use some computer boards that do not have a Wi-Fi chip and therefore we have to use additional hardware for communication with such boards. To keep this communication as simple as possible, we use an external Wi-Fi module, which can itself be easily controlled. In order to keep the amount of the extra hardware required as low as possible, we have connected all the boards that do not have Wi-Fi connectivity to a single board that has a Wi-Fi module, which in turn allows for a simple software implementation for the Wi-Fi communication. The software complexity is thus directly proportional to the hardware complexity in this case.

3.1 Experimental Setup of the Proof-of-Concept System of Systems

For the PUF implementation that serves as a security anchor, we use a Raspberry Pi 3B+, which exchanges data with another device that is part of the SoS. The DRAM PUF being utilised is discussed in Section 3.2. We are also using two NodeMCU LUA Amica V2 development boards, each with an ESP8266 ESP-12E Wi-Fi module, which can communicate data directly via their built-in Wi-Fi chips. Also, we are using two Olimex ESP32-EVB boards, which do not have a built-in Wi-Fi connection, and one Olimex ESP32-EVB-EA board, which has an external Wi-Fi Antenna. The two ESP32-EVB boards communicate with the ESP32-EVB-EA board via CAN; the ESP32-EVB-EA board acts basically like a gateway for the ESP32-EVB boards to access the rest of the network. Since our SoS is based on the master-slave principle, we do not need to exchange keys between all devices, but rather only between the master device and the slave devices. In this case, the master device knows the keys of all slave devices. Therefore, potentially untrusted devices can only communicate with the master device, which knows about all other devices and manages the data traffic. This configuration setup can be easily changed through a configuration file included in our implementation, such that the devices can communicate with each other as well. An overview of the complete SoS can be found in Figure 1. As is evident, the SoS implemented is heterogeneous in terms of both the devices and the communication protocols being utilised, in order to better emulate a generic SoS case. Finally, our implementation also includes a number of sensors and actuators, not only in order to provide a more realistic proof-of-concept prototype of an IoT SoS, but also to test and validate the flexibility and scalability of the security solution being proposed on real data. Currently, light, and temperature and humidity sensors are used, which could be replaced, depending on the use case, by proximity sensors, lidars, radars, cameras, and so on. Accordingly,

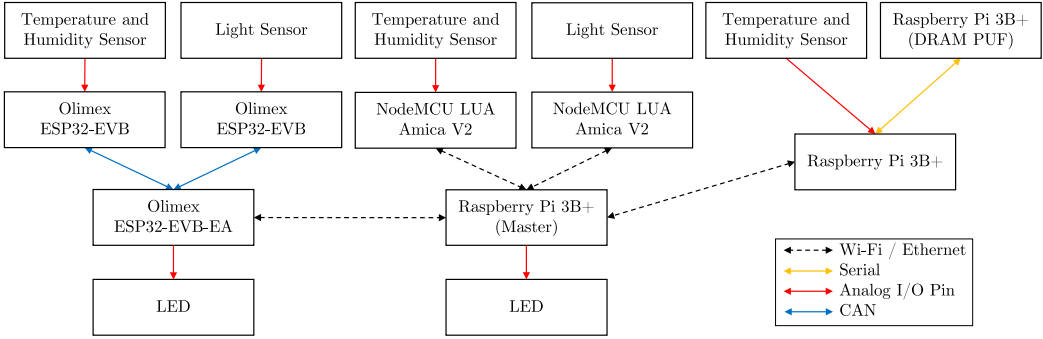


Fig. 1. Experimental setup of the implemented proof-of-concept prototype of a **System of Systems (SoS)**. The SoS is heterogeneous in terms of both the devices and the communication protocols being utilised.

in our setup, only LEDs are currently employed as actuators due to their simplicity. These can, of course, also be replaced by other devices like motors, relays, and so on.

3.2 The Physical Unclonable Function Utilised

The PUF utilised as the security anchor of our implementation is provided by the DRAM of a Raspberry Pi 3B+ development board. The board has a Broadcom BCM2837B0 quad-core **System on a Chip (SoC)** module and a Broadcom Videocore-IV GPU. The board also uses a 1 GB LPDDR2 SDRAM as its main memory. In this article, the decay feature of the SDRAM is being used as a runtime accessible DRAM PUF. Our DRAM PUF implementation is based on the work discussed in [5] and [7].

In the Raspberry Pi 3B+, only the GPU has the privilege to control the register controlling the SDRAM self-refresh operation. Therefore, our DRAM PUF code is executed in the GPU which is physically isolated from the CPU. The GPU and CPU can communicate with each other via the Linux mailbox framework. There are three parameters that are relevant to our DRAM PUF implementation: *PUF_address* is the SDRAM address that should be used as the DRAM PUF. This address should be reserved both in the kernel and the firmware to make sure that the other code being executed does not write to or read from this address. Obviously, a set of SDRAM addresses can be utilised simultaneously in the context of the *PUF_address* parameter being a set of memory addresses, in order to increase the entropy of the PUF response. In the DRAM PUF query process, *decay_time* is the amount of time that the SDRAM refresh should be disabled. The *decay_time* and *PUF_address* should be selected according to the decay features and the working environment of the DRAM chip, in such a way that the DRAM PUF generates enough bit flips to ensure that the produced secret key has enough entropy. *initial_value* is the data that is used for the initialisation of the *PUF_address*. To ensure the security of the key generation scheme, the parameters are recommended to be compiled within the firmware of the Raspberry Pi 3B+.

In the PUF query process, the memory controller is taken over by the PUF query code running on the GPU. First, the self-refresh of the whole DRAM chip is disabled. Meanwhile, the operation of other relevant code should be suspended, to prevent potential security threats, e.g., operating a Rowhammer attack [33] by repeatedly reading SDRAM addresses physically close to the *PUF_address*. To prevent data loss, the SDRAM addresses that store meaningful data need to be refreshed periodically by issuing a manual read operation to the first word of each row of such addresses. The manual refresh period should be less than the retention time of the SDRAM module, which is equal to the time that the weakest DRAM cell can reliably hold data. Usually, the minimal

retention time is 32 ms. Therefore, in our implementation, we manually refresh any essential data stored on the SDRAM, during each 2 minutes of decay time at least $120,000\text{ ms} \div 32\text{ ms} = 3,750$ times. After a time period equal to *decay_time*, the self-refresh is recovered by the code running on the GPU. The DRAM PUF data stored on the *PUF_address* are now maintained by the self-refresh functionality of the SDRAM module. The code running on the CPU can read the DRAM PUF data directly via the memory controller from *PUF_address*, and then generate the secret key by utilizing a bit selection/correction scheme. This key can be transferred to and used by another device through the UART module and the code running on the CPU of the Raspberry Pi 3B+ that is used for the implementation of the DRAM PUF.

The detailed DRAM PUF query process in our proof-of-concept implementation is as follows: First, the relevant parts of the DRAM are written fully with logical zeros (or ones), a bit pattern that was referred to as the *initial_value* before. Next, the periodic self-refresh operation of the DRAM cells, which is controlled by the memory controller, is switched off. Then, a *decay_time* of 2 minutes needs to elapse, during which the values stored on the relevant DRAM segments do not remain stable and some bit flips occur, because the periodic self-refresh operation of the DRAM has been switched off. However, due to minor manufacturing variations, some DRAM cells lose their charge faster than others. During a one-time enrollment phase, this experiment is carried out several times and the whole DRAM is dumped every time. All these memory dumps are then compared programmatically and all “stable” addresses are gathered. A cell is considered “stable” if its value has remained the same in a sufficiently large number of experiments. Assuming that the initial value of the cells is a logical zero, such a cell can either have the value of a logical zero (stable value retention) or of a logical one (stable bit flip) at the end of each experiment. Due to the characteristics of the DRAM and the short *decay_time* being used, there are, of course, far fewer (stable) bit flips than cells with a stable value retention. To extract a cryptographic key from the PUF response, either a fuzzy extractor scheme can be used, which has the advantage of being error-correcting, or certain stable cells (comprised of a permutation of cells exhibiting stable bit flips and stable value retention) can be selected, with the concatenation of their values then constituting the key. In our implementation, the latter scheme is used to extract 128 bits, which then represent a key that is 16 bytes long.

3.3 The Proposed Protocol

In order to utilise a highly scalable and lightweight data transmission protocol that offers a rather simple implementation, we have opted to use the MQTT protocol, which offers different “topics,” which clients can subscribe to in order to receive messages sent to them. In this way, client devices not only can exchange information related to the current system time with a server (broker), in a network configuration that follows the master-slave principle, but can also directly provide sensor data to the broker and other client devices. More specifically, the clients can provide measurement data from their sensors both to other client devices, which can take appropriate actions that may be realised using the relevant actuators, and to a server (broker) that receives, processes, and shows the data over a web interface that can be accessed by outside devices, as well as potentially re-transmitting these data to other brokers for which such data may be relevant.

In order to exchange data, we defined a protocol, which ensures the confidentiality, authenticity, and integrity of the sent data. To achieve these goals, we are using the HMAC-SHA-256 hash algorithm to authenticate the packets sent, and the AES-128-CBC encryption algorithm to encrypt the data. Since the protocol is based on a shared key as the encryption algorithm is symmetric, the protocol requires a one-time enrollment and key agreement phase before the devices can communicate with each other. This enrollment and key agreement phase should take place in a secure environment or utilise symmetric key exchange strategies such as the Diffie-Hellman key

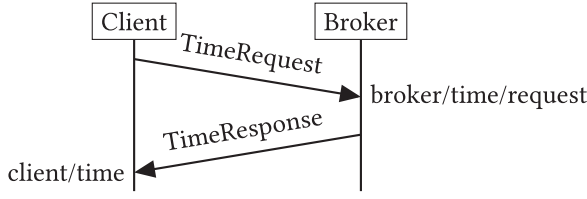


Fig. 2. Outline of the proposed time protocol.

1	1-255	8	32
Address Length	Address	Nonce	HMAC

HMAC-SHA-256 authenticated

Fig. 3. Outline of the TimeRequest.

agreement scheme. The key generated by the procedure mentioned in Section 3.2 must be communicated to all relevant devices. If the devices do not or should not trust each other, multiple keys can be generated from a single PUF response, or multiple PUF responses from different addresses in the DRAM could be collected, such that each device can be assigned a different key. In our implementation, all devices trust each other, so we extract only a single key from a single PUF response. The devices now know about each other and can exchange data. However, since the received data cannot be correctly assigned due to the limitations of the MQTT protocol, each device must also send its address, which is unique for each device in the SoS. In addition, a common time system must be established within the SoS, as the devices may not have any access to a real-time clock. Thus, to keep the notion of time the same on all devices, and thus keep them synchronized, the broker sends the current system time to each client at its request. Therefore, the time kept by each individual device is almost identical and the data can also be correctly timed. This also allows us a certain flexibility, as we can potentially not use the time system of the broker, but a time server, which may be external or internal to the SoS, as the synchronization source. However, small fluctuations in the millisecond range will anyway occur, as the devices need a small amount of time to decode, encrypt, and exchange the relevant synchronization messages. Nevertheless, this can be balanced out by additional messages acting as a ping test. In our implementation, a received message is simply rejected/ignored if its transmission time has been more than 5 seconds ago, or somehow appears to be in the future. This time value is selected arbitrarily and can be adapted to the respective use case. Figure 2 shows the dataflow during time synchronization.

As one can see, the client sends a request, called TimeRequest, to the broker, and the broker then sends his response, which includes the current system time and is called TimeResponse, to the client. These packets are described in detail in Figure 3 and Figure 4, respectively. Also, neither the TimeRequest nor the TimeResponse are encrypted. Therefore, even if an external time server is used, it does not need to know the encryption key used. In our implementation only the standard local time is used, so the time-related information constitutes no secret. In other environments, the time might differ from the standard time and, therefore, the relevant messages may have to be encrypted, depending on each particular use case.

In Figures 3 and 4, the number above each section of the relevant packet indicates its size in bytes. The “Address Length,” “Address,” and “Timestamp” sections have already been discussed. The “Nonce” section is generated randomly each time in our implementation, and can be used to further authenticate the protocol. We also use the nonce to better classify packets that may

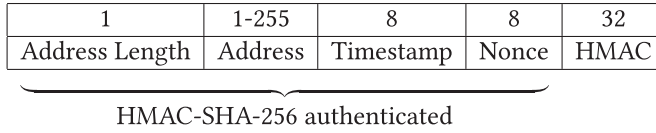


Fig. 4. Outline of the TimeResponse.

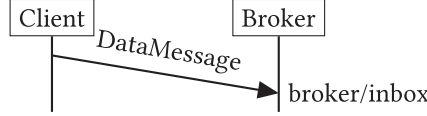


Fig. 5. Outline of the proposed data protocol.

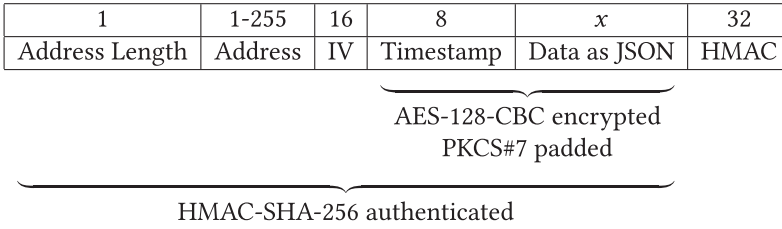


Fig. 6. Outline of the DataMessage.

arrive at the same time, or even arrive twice, due to a potential misconfiguration of the **Quality of Service (QoS)** property of the MQTT. It can happen that two TimeRequests reach the broker or two TimeResponses reach the client at the same time and therefore the timestamp could be wrongly set. Thus, the nonce can correctly classify the first and second packet and identify the old timestamp, in order for it to be ignored. These packets are exchanged every 5 seconds to also continuously serve as keepalive packets. This continuous synchronization allows the information about the current time to be included in the exchange of data. The general data protocol is shown in Figure 5.

The packet DataMessage, which is only used to exchange data, is explained in more detail in Figure 6.

Again, in Figure 6, the number above each message section show its size in bytes. Additionally, the “Address Length,” “Address,” and “Timestamp” sections have already been described. The “Data as JSON” section is composed of the message data in the JSON data interchange format. This allows us to collate data and send more data at the same time. The “IV” section is the initialization vector that is used for the AES-128-CBC encryption with PKCS#7 padding and, in our implementation, it is regenerated each time a message needs to be exchanged. The secret key for the AES-128-CBC encryption is the key exchanged in the enrollment and key agreement phase, which has been generated using the PUF. Since both parties know this key and the content of the “IV” section, the “Data as JSON” and the “Timestamp” sections can be encrypted and decrypted, by the client and the broker, respectively.

The complete protocol can be seen in Figure 7. To exchange data successfully, all devices must maintain a synchronized time system. Therefore, in general, each time a client wants to start communicating, it simply needs to send a TimeRequest. Then, the broker responds with a TimeResponse which is processed by the relevant client, which then synchronizes its time with the locally maintained time system. After this synchronization, the client can send as many

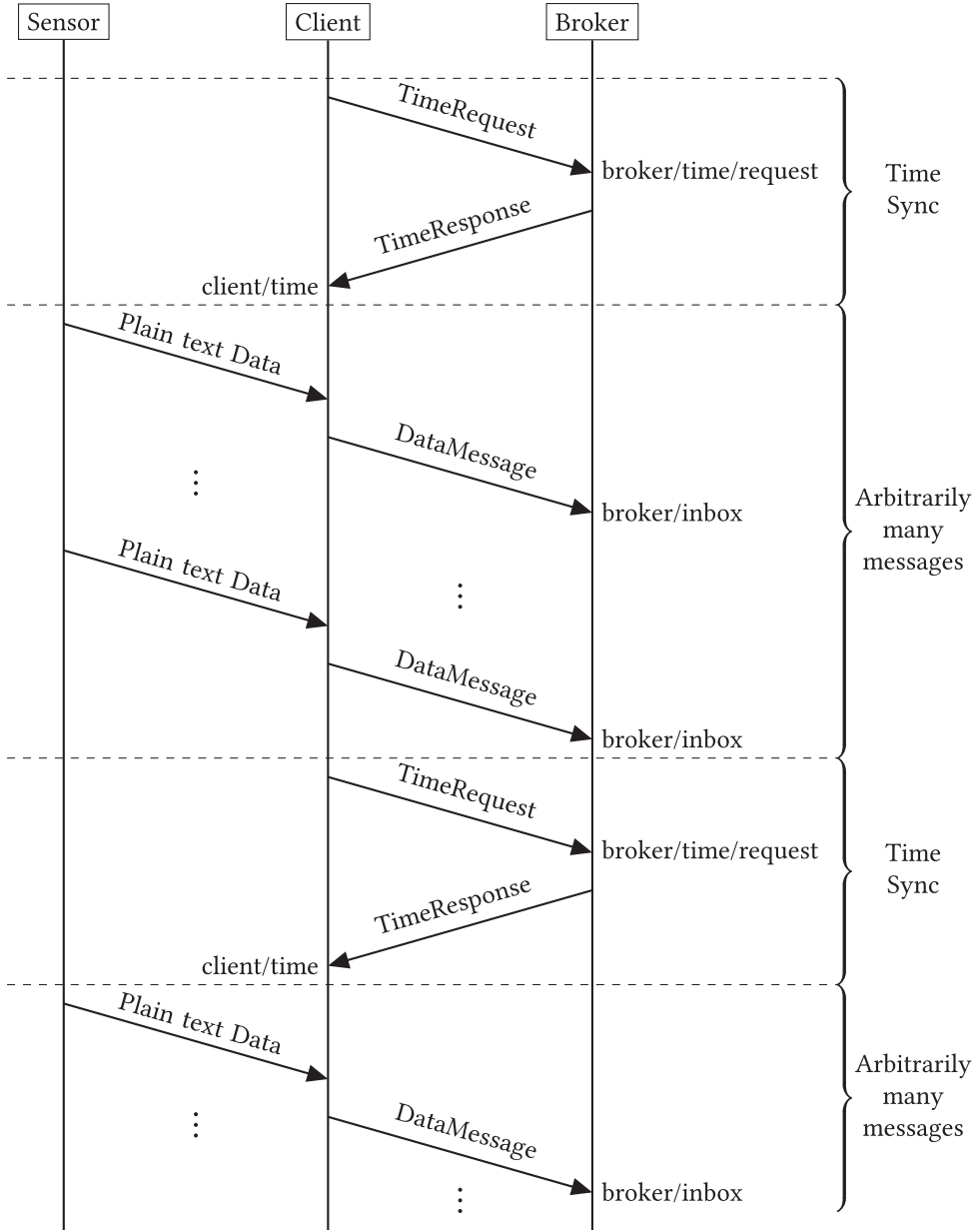


Fig. 7. An overview of the proposed protocol.

DataMessage packets as it wants. In Figure 7, we show the client sending DataMessage packets to the broker, only as an example of data communication, as a client can send DataMessage packets also directly to other client devices, as long as they are sharing the same key. In our implementation, client devices that are subscribed to a particular “topic” of the MQTT protocol form a group and share a common key. Moreover, since packets are rejected in our system if they are older than 5 seconds, the time may therefore need to be resynchronized every 5 seconds, which is why another TimeRequest will need to be sent, as shown in Figure 7. After the TimeResponse has been

processed by the client, it can send again as many `DataMessages` as it needs to, for a time period of at least another 5 seconds. This process is repeated periodically until the system is shut down.

Finally, we see that our solution is lightweight and scalable due to our use of a low-cost hardware security primitive, i.e., a PUF, in combination with lightweight software security algorithms, to secure the system. In the emerging era of systems of systems, this co-engineering of hardware and software allows for widely varying applications as it does not require as many resources as other already proposed solutions may need, thus enhancing the novelty of our security scheme. Therefore, we conclude that the novel security architecture framework that we propose for the present and the next generation of the IoT in the context of the emerging era of SoS can provide an advanced level of security, while still being highly scalable, flexible, and lightweight.

3.4 Our Security Solution in the Framework of IoT Systems of Systems

In this subsection, we examine the proposed security solution in the context of different systems of systems. For example, we note that our solution could be applied to the use case of a smart vehicle, which forms on its own a system of systems. In order to use our protocol, we need to adapt the devices from our proof-of-concept implementation to the different sensors and actuators in such a system. The broker can be placed into a smart car as-is, because its only purpose is the creation, management, and distribution of timestamps. In order to add more security to the system, as will be discussed later on in Section 5.4, some fallback brokers may also be added.

Additionally, in this use case, some of the relevant sensors would be, e.g., proximity sensors, or even the tachometer. On the other hand, some of the relevant actuators would be the lights, steering wheel, or even the brakes. We therefore easily note that since some of the relevant devices, e.g., the steering wheel and the lights, do not have to (and, thus, should not) communicate with each other, they should have no direct communication channel. This can be accomplished by simply having no shared encryption keys. Also, the other transmission keys should be pairwise/groupwise different from each other so that one compromised key cannot compromise the whole network communication, or even the communication between other pairs/groups.

Essentially, as we will discuss in Section 4, this use case can benefit from the introduction of multiple brokers that would serve smaller segments (which are highly independent from each other) of the overall system of systems. To this end, we could divide the in-vehicle network into different communication sectors and layers. This would improve the performance of such a system of systems, as well as the scalability of our security solution. An example of such a subdivision could include the locomotion, interior fittings, central locking system, and so on, as communication sectors. These subsystems will then communicate via another abstract layer through either their corresponding brokers or additional dedicated network devices. In general, our proof-of-concept implementation should be able to adequately simulate each of the different sectors and layers of such a system of systems.

Moreover, the proposed security solution can also be applied to agriculture, smart grids, and even smart homes. Possible candidates for sensors in these use cases could be, e.g., crop growth or weather sensors, electricity meters, or even cameras. Similarly, possible actuators may include, e.g., irrigation systems, heaters, autonomous windows, autonomous emergency units, or actuators of such systems of systems as harvesters. Therefore, also these use cases may benefit from the introduction of multiple brokers that would serve subsystems of the system that are independent from each other.

We note that the described use cases form examples of how the proposed security solution framework can be applied to individual IoT applications in the context of systems of systems. Of course, the proposed proof-of-concept system-of-systems implementation should be modified and/or expanded accordingly, in order to be able to adequately simulate each different use case considered,

in terms of the devices and network protocols being utilised. Nevertheless, we believe that our solution is a generic security solution that can be applied to different use cases where highly heterogeneous devices and networks are combined in an ad hoc manner, and that the concept of hardware and software security co-engineering that our work is based upon will eventually form the standard for security in the field of systems of systems.

4 BRIEF EVALUATION OF THE PERFORMANCE OF THE PROPOSED SOLUTION

The previous section presented a lightweight and therefore low-cost security solution for modern SoS. Thus, we can conclude that, in general, our security solution appears to be applicable to most, if not all, IoT use cases in the context of systems of systems. As already mentioned in Section 2, which considers the related work, other works have so far not been able to address the security requirements of IoT systems in the emerging era of systems of systems as effectively as our security solution does.

In general, our solution benefits both from hardware and software security co-engineering, as it combines an intrinsic hardware security primitive with well-known software algorithms for cryptography, in the context of a robust and cost-efficient protocol, and from employing an already widely established data exchange protocol (MQTT) that has proven to be highly efficient. Since PUFs are a cheap alternative to many security mechanisms, we chose to use the weak intrinsic DRAM PUF of a Raspberry Pi 3B+ to keep the implementation and operation costs as low as possible. Since the implementation of Chen et al. [5] is based on the aforementioned rpi-open-firmware project, which cannot boot a complete Linux Kernel yet (although there are project forks like librerpi's firmware project,³ which seem very promising regarding future work on this topic), we need to use another Raspberry Pi 3B+ for all PUF-related procedures. This extra cost can, as already mentioned, be eliminated by a firmware and/or a kernel that allow accessing specific features of the DRAM while still providing all the benefits of a full-functioning operating system.

Also, the PUF comes at another cost. Its stable addresses must be determined, which takes significant time during the one-time enrollment process. Given that a PUF response only emerges after a certain decay time, during which some of the DRAM cells are leaking some of their charge, also the generation of each PUF response requires some time. Therefore, the potential caching of the PUF response may make sense in some use cases, but it comes at a security cost, as it obviously allows for, e.g., side-channel attacks against the cached secret.

Encryption and hashing also come with extra costs. Since AES-128-CBC requires an **Initialization Vector (IV)**, this IV needs to be either included in the message itself, as is done in our implementation, or be static and exchanged beforehand. Therefore, there may be a need for an extra 16 bytes per data message. Time synchronization messages do not need encryption in our SoS, but could also be affected by this, as will be further discussed in Section 5. Additionally, the padding also takes up some space in most cases. Also, the HMAC result needs to be transmitted, which occupies another 32 bytes per message. Furthermore, the nonce used in time synchronization uses an extra 8 bytes per message. Apart from that, the system does not require extra messages, or anything similar to function properly.

Moreover, the encryption and decryption processes also require resources and time. Depending on the microcontroller and programming language used, this can vary by a significant amount. In our case, we used the Go programming language on the master Raspberry Pi 3B+, Java on the Raspberry Pi 3B+ with a sensor, C/C++ on the Raspberry Pi hosting the DRAM PUF, and C/C++ using the Arduino IDE on all the other evaluation boards. Neither the extra time nor the extra resources needed for the communication process affect the system in a significant way. However, it

³<https://github.com/librerpi/rpi-open-firmware>.

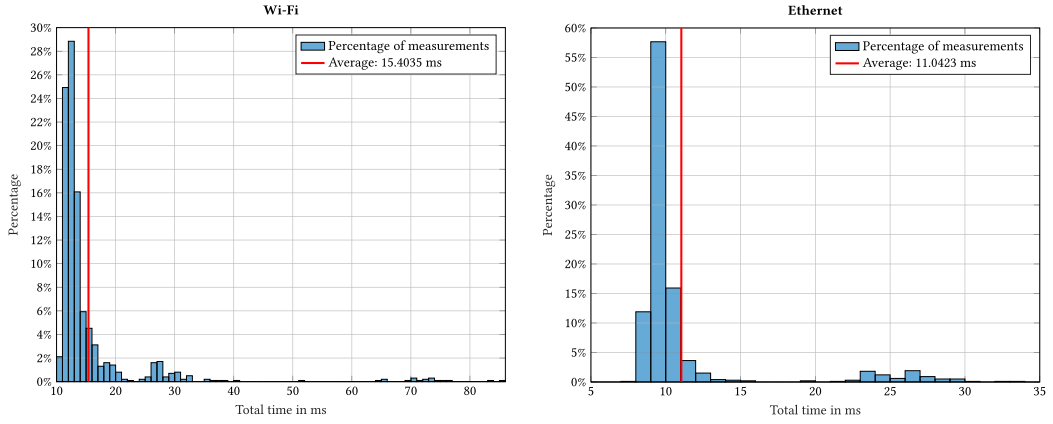


Fig. 8. Measurements of the total time (**Round Trip Time (RTT)** + **Packet Processing Time (PPT)**) required to send a message and receive a response, when either Wi-Fi (left sub-figure) or Ethernet (right sub-figure) is used.

should be noted that the resources and time required could play a more significant role in systems with even more limited resources or more exact time requirements. By using different programming languages, we have also shown the security solution proposed can be implemented in a heterogeneous set of devices utilising different software code, and that all potential software requirements of our solution, such as software libraries related to the MQTT protocol, or software implementations of the relevant cryptographic algorithms can be easily met, even when different microcontrollers and varying programming languages are utilised.

In order to experimentally examine the performance of our proof-of-concept SoS implementation, we have measured the overall time required for a message to be encoded, sent, processed by the receiver, and responded to. Since our system supports different transmission methods, we needed to evaluate them independently from each other, because their **Time of Flight (ToF)** might differ significantly. When measurements are taken using Wi-Fi, the devices are placed 10 cm apart from each other, while when Ethernet is used, a LAN cable with a length of 1 m is utilised. In Figure 8, we can see the overall time a message and its response need (in milliseconds) in order to be sent and processed, i.e., the sum of the **Round Trip Time (RTT)** and the **Packet Processing Time (PPT)**, in case either a Wi-Fi or an Ethernet connection is used. One can note that unsurprisingly Ethernet with an average of 11.0423 ms is, in general, faster than Wi-Fi with an average of 15.4035 ms. It is also worth noting that the variance of all measurements using Wi-Fi is approximately 83.5339 ms and the variance using Ethernet is approximately 20.7333 ms, indicating the relative instability of the Wi-Fi connection, which can be caused by various external factors. Depending on the requirements of each use case, the achieved communication time can be considered as an indicator of real-time communication, as the overall time required for RTT and PPT is in almost all cases less than 30 ms, and most often less than 16 ms. This implies that the results of our SoS prototype are fully applicable in such real-time applications as in-vehicle communication for autonomous vehicles, as indicated by the timing analysis provided by the **AUTomotive Open System ARchitecture (AUTOSAR)** consortium for real-time applications.⁴

It should also be pointed out that, although not tested, our system should also be compatible with the LoRaWAN and ZigBee communication protocols, with some minor adjustments. Both

⁴https://www.autosar.org/fileadmin/user_upload/standards/classic/4-2/AUTOSAR_TR_TimingAnalysis.pdf.

technologies are often utilised in the communication of modern IoT systems, and therefore there are already many software libraries, like ZigBee2MQTT,⁵ that can help provide a bridge between the MQTT protocol and these communications protocols.

In addition, it should be noted that in our case the broker is the main performance bottleneck in the overall system, since it communicates with all other devices. In large networks, this can become a major performance problem. To overcome this obstacle, the SoS can offer multiple brokers that manage subsystems of the SoS that are highly independent from each other. In this case, the SoS would be segmented into different communication sectors and layers, as discussed in Section 3.4. Each of these brokers would then communicate only with the subsystem devices relevant to it, and with the other brokers in the context of the overall SoS, thus limiting the number of connections required and, therefore, also the number of symmetric keys and PUF responses needed. On the other hand, the system could also potentially use a different network layout and topology, other than the master-slave principle employed in our system. For example, instead of using one or more brokers, a **Peer-to-Peer (P2P)** network could be implemented. This way, even more performance benefits could potentially be gained.

In general, we note that the proposed solution is flexible, due to its many configuration possibilities and its extensive compatibility with different devices and network protocols, as well as lightweight and cost-efficient, due to its extremely limited use of both software and hardware, as we have examined in this section. We can reach this conclusion, because our protocol in theory does not require the addition of hardware—apart from the device hosting the DRAM PUF, which can be removed as soon as the conditions discussed in Section 4 are met—and only limited additional security data are sent per message, while also the utilisation of internal resources, such as the CPU, is also limited. Therefore, the proposed security solution can be supported even by very resource-constrained devices, which are often employed in the IoT due to their very low cost, which is further reduced due to the production of such devices under economies of scale. Moreover, we also observe that the proposed security architecture framework is highly scalable and configurable, while providing adequate security, and, therefore, is a highly suitable security solution for current and next-generation IoT devices and networks in the emerging era of systems of systems.

Finally, we note that our key distribution system is based upon the establishment and sharing of keys among groups of devices, which is the most realistic approach to offer authenticated keys in such an SoS, as it provides the best tradeoff between security and performance, e.g., in such use cases as in-vehicle networks [28].

5 DISCUSSION OF THE SECURITY OF THE PROTOCOL PRESENTED

In the development of IoT devices, the aspect of security is often neglected for reasons of cost and time. Initially, it may seem unimportant if outsiders have access to an IoT network, e.g., in the home automation sector, and can view certain measurement data, e.g., how high the current temperature is in the living room of a house. But, nowadays, IoT systems can also control the elementary electronics of the household and, therefore, successful attacks can cause rather significant damage. For example, if the values of a temperature sensor are manipulated due to weaknesses in the security of a smart home SoS, shower users could be scalded due to the potential misuse of the smart water heater.

In the context of systems of systems, attacks that can affect the content of the messages being exchanged, such as message manipulation or replay attacks, are critical, as they can have an extensive effect on a wide range of the system's subsystems. Therefore, we have paid particular attention to such attack vectors, in order to secure our system against them. We do this by setting

⁵<https://github.com/Koenkk/zigbee2mqtt>.

up an attacker model in the context of the following three security objectives: (1) Confidentiality, (2) Integrity, and (3) Availability, which together comprise the well-known CIA triad. Our attacker model is based on the CIA triad, in order to classify and regulate the expected capabilities of potential attackers. Against this background, we have drawn up a list of common attack vectors (both for the devices and the networks employed in the SoS) that need to be fended off.

The CIA model utilised in our work shows which of the three security concepts, i.e., the confidentiality, integrity, and/or availability, of the SoS a particular attack targets. More specifically, the protection goal of confidentiality is violated as soon as attackers with access to the system are able to read the content of messages. If attackers are able to manipulate existing messages, or forge and send their own messages, the message integrity will have been successfully compromised, as long as the affected messages are considered as legitimate by the system. Such a successful attack would allow for significant control of the operation of the SoS. For example, forged sensor data could be transmitted, in order to trigger undesirable actions, e.g., switch off an alarm system. Finally, availability is important in order to ensure that the relevant system of system functions continuously. A rudimentary attack on availability would be, for example, to prevent communication by disconnecting some of the SoS communication cables. Additionally, it is also possible to disrupt Wi-Fi communications through a **Denial-of-Service (DoS)** attack.

5.1 Attacker Model

Our attacker model is described through the example use case of a secure smart home, i.e., an ordinary home with a heterogeneous smart IoT network, consisting of both wireless and wired devices and modules. A master device records the measured values of the sensors from the house and controls some of the smart home actuators. The network includes a CAN bus for controlling fixed installations, such as lighting and heating, and an additional Wi-Fi network for the wireless smart home devices, e.g., a vacuum cleaner robot. Communication keys have already been negotiated during a one-time enrollment and key agreement phase and are therefore not sent again within the SoS.

We define the following preconditions and assumptions in our attacker model. We assume that the attackers are private individuals with correspondingly limited time, limited financial resources, and limited computing capacity. We do not rely on the security of the network protocols used, which is why we assume that attackers already have access to all network segments, notwithstanding the protocol being used. In particular, attackers are assumed to have the ability to eavesdrop on the communication of the Wi-Fi network even outside the home. It is, however, assumed that there is no physical access to parts of the system of system other than the communication channels. This means that attackers cannot manipulate, e.g., the microcontrollers in a smart fridge, but can read the messages transmitted via the cables connecting the different microcontrollers. Beyond that, no other elements are under the control or observation of the attackers.

In this way, it is ensured that both the PUF and the keys that are produced from its responses cannot be accessed by a potential attacker, as this would require either physical access to the device or privileged software access to it. Therefore, our security model is rather focused on the security of the communication among the nodes of the SoS. In general, the competence of a PUF-based security solution for the IoT has already been extensively examined and is considered as proven in the relevant literature, which we have discussed in Section 2.

5.2 Mitigated Attack Vectors

The attacker considered in this work essentially adheres to the Dolev-Yao model [11], i.e., the attacker is able to manipulate the exchanged messages in any way apart from breaking the relevant cryptographic primitives. In the list that follows, we provide a set of attack vectors that our security

solution can mitigate, each followed by the relevant CIA component being affected. Furthermore, we briefly describe the effect of each attack vector in the use case of a smart home for better understanding. The way in which these attacks can be prevented is discussed in Section 5.4.

- (1) Attackers can intercept a message and read the user data in plain text (confidentiality).
In the context of a smart home, intercepted messages can contain, for instance, the readings of a temperature sensor, which would be exposed to the attacker.
- (2) Attackers can manipulate the content of a message (integrity).
Here, the target temperature of the heating system could be manipulated, so that room temperature, for instance, is increased above the intended value.
- (3) Attackers can send a message pretending to be another sender (integrity).
A smart refrigerator that only orders new items when they are used up would, in this case, order items that are already in store.
- (4) Attackers can create and send their own messages (integrity).
This way, in a smart home, lights can be turned on or off without this being desired by the user.
- (5) Attackers can delay the delivery of a message (availability).
A smart stove that sends a message to the user's smartphone when the relevant timer has expired would, in this case, not notify the user when the food is ready, and food could be served and consumed cold.
- (6) Attackers can duplicate a message (integrity).
Messages exchanged between the house's smart lock and the user's key could be stored and used to enter the house when the user is away.
- (7) Attackers can interfere with the broker (availability, integrity).
In a smart home, the broker can be shut down and therefore become unavailable for sending and managing timestamps for the other devices, e.g., smart fridge, smart TV, and so on.

5.3 Unmitigated Attack Vectors

We will not address the following attack vectors, mainly because providing sufficient protection against them would be too costly and/or too complex:

- Attackers can observe the Wi-Fi data traffic (confidentiality): By observing the size and frequency of the sent data packets, conclusions can be drawn about the confidential information that these packets contain, even without knowledge of the content. For instance, by observing the commands sent to the light switches, an attacker could infer whether a user is at home or not.
- Attackers can carry out global Dos attacks (availability): Such attacks would prevent the exchange of messages within the network, by overloading the communication channels, jamming the relevant frequencies, and/or disconnecting or destroying particular wires, cables, and/or devices. In this case, a wireless smart lock may, for instance, deny a user entrance to the smart home as the key information can no longer be received by the lock.
- Attackers can take over a sensor (availability): With access to the premises, sensors, and actuators can be manipulated on site (e.g., motion detectors can be taped over). In addition, by gaining access to the sensor, it may be possible to generate false sensor data that may be considered as legitimate (integrity).
- Attackers can take over physical components of the SoS (availability): If the SoS is physically accessed, or privileged access is gained to it, it becomes extremely vulnerable to manipulation, as even the DRAM PUF responses that form the security anchor of the system may

potentially be exposed. In our implementation, this could happen if the device hosting the DRAM PUF is taken over.

5.4 Plausibility Check of the Proposed Security Protocol

Here we describe how the attacks discussed in Section 5.2 can be addressed by our security solution.

5.4.1 Attackers Can Intercept a Message and Read the User Data in Plain Text. Meaningful user data are encrypted using the AES-128-CBC encryption algorithm with PKCS#7 padding and therefore are not readable. The device addresses and the content of the time messages (messages for synchronising the clocks, since the client devices may not have real-time clocks) are transmitted unencrypted, as they are not meaningful user data, but are, in either case, authenticated using the HMAC-SHA-256 algorithm. The device addresses must be transmitted unencrypted so that the receiver can use the key that matches to the sender for decryption. The time messages only contain the current system time as maintained by the broker, which is not considered as secret information (as system time is kept updated by the broker's operating system using information from a public **Network Time Protocol (NTP)** server) and, thus, no encryption is necessary.

5.4.2 Attackers Can Manipulate the Content of a Message. The integrity of all messages is ensured by an HMAC (in particular, the HMAC-SHA-256). If a message is manipulated, the HMAC no longer matches the content of the message and the message is rejected by the recipient.

5.4.3 Attackers Can Send a Message Pretending to be Another Sender. The authenticity of the messages is also ensured via the HMAC. The attacker does not know the secret HMAC key between the alleged sender and the recipient and, therefore, cannot generate a valid MAC.

5.4.4 Attackers Can Create and Send Their Own Messages. If the attacker selects a device address unknown to the pre-existing network, the messages containing this device address are immediately rejected by their recipients, since device addresses cannot be assigned dynamically, after the enrollment and key agreement phase. Otherwise, the attacker must reuse a foreign device address, i.e., send messages in the name of another sender. This is prevented as already explained.

5.4.5 Attackers Can Delay the Delivery of a Message. The delivery of messages cannot be guaranteed, since the attacker can prevent any communication between two devices and no information can be transmitted in this case. Data messages contain a timestamp and are discarded if they are too old. Time messages are exchanged according to a request-response scheme. The client sends a request with a randomly generated nonce to the broker, which then replies with a response containing the nonce and the current time. The client then checks that the nonce received matches the one sent and potentially also that not too much time has passed between the request and the response. If the nonces do not match or a significant delay is detected, the corresponding messages are ignored.

5.4.6 Attackers Can Duplicate a Message. The timestamp in the data messages can be considered as a counter. A message must always have a newer timestamp than the previously received message. This does not apply to a duplicated message and, therefore, only the original message would be accepted and the duplicate would be discarded. If a TimeRequest message is duplicated, the broker answers again with the current time. This results in a repeated TimeResponse message. A filtering of duplicate TimeRequest messages is currently not implemented in the broker, since creating the TimeResponse messages is rather trivial. A duplicate TimeResponse message would be ignored by the client, since a TimeResponse message must always correspond to a TimeRequest one (based on the transmitted nonce). In general, the TimeRequest message is marked as processed

and discarded by the client as soon as the original TimeResponse arrives and, therefore, the duplicate TimeResponse message would be discarded.

5.4.7 Attackers Can Interfere with the Broker. If the broker becomes unavailable, its sole purpose, which is to manage and transmit timestamps, would be disrupted. As a result, messages could no longer be secured and classified via timestamps. Accordingly, replay attacks would become possible as a result. To counteract this, fallback brokers can be added to the system, which can switch on as soon as the broker is no longer available for a certain amount of time. These alternative brokers should, of course, have the same functionality as the broker, so that they can also generate, manage, and distribute timestamps. A potential failure of the broker can be detected by observing the handshake/keepalive packets (the ones that contain time messages), to see if it is still available. In this way, also inactive/unavailable devices can be detected. In the case of a real-time system of systems, fallback brokers may not be able to switch on in time. However, since only the communication between the broker and the clients is affected, the individual client devices in the network can still communicate with each other. While the broker is unavailable, replay attacks may be possible, but no impersonation attacks.

Furthermore, the keys within the direct communication among client devices are different from those in the communication between each client and the broker, so compromising the keys stored in the broker does not result in all communication in the network being decrypted. If the broker is compromised, attackers could impersonate the broker and potentially disrupt the network communication by providing invalid timestamps, and potentially causing a widespread DoS. However, no meaningful user data could be compromised. Additionally, as a DRAM PUF response can offer multiple keys, in case some of the keys are compromised, they can be instantly replaced, when this is noticed, and therefore such an attack is not really scalable.

6 CONCLUSION

In this work, we have observed that a new generation of the IoT is emerging and that systems of systems are becoming increasingly relevant. As the IoT has found application in a wide variety of areas, systems, and architectures, we have presented a proof-of-concept implementation of a heterogeneous state-of-the-art SoS, which employs diverse communication protocols. However, we have also noted that security remains an open issue for systems of systems, because of the heterogeneity of their components and the ad hoc formation and utilisation of rather complex systems of systems. In order to address this, we have implemented a lightweight, flexible, practical, and scalable security solution for the presented SoS implementation.

More specifically, the proposed security solution is based on a novel hardware and software co-engineering paradigm, combining state-of-the-art software and hardware security mechanisms by design. In this way, we aim to utilise the advantages and mitigate the shortcomings that software and hardware security mechanisms have on their own. In particular, our solution is based on symmetric cryptographic algorithms in software, which, however, utilise a DRAM PUF as their security anchor. Hence, our solution forms part of a novel security architecture paradigm for the IoT that combines hardware and software security primitives, while at the same time remaining lightweight and cost-efficient.

Moreover, in Section 2, we have presented and discussed a number of related works and shown that our solution brings about a number of advantages over the existing solutions. We have also noted that a design similar to our solution had not yet been proposed and examined in the context of systems of systems. Our proof-of-concept implementation is discussed in Section 3, where its setup, which is based on commercial off-the-shelf components, is described. In the same section,

we also examine the PUF that is employed as a security anchor, as well as the software security algorithms employed, and the overall security protocol being proposed.

Furthermore, in order to assess the proposed security solution, we have shown, in Section 4, that the overall time needed to establish security is almost always lower than 30 ms and, thus, the proposed security protocol should be applicable even to real-time applications. We have also shown that the implementation of the protocol does not bring upon high additional costs, with regard to both the hardware and the software being required, as the security solution proposed is rather lightweight, scalable, and flexible.

Additionally, in Section 5, we have proposed an attacker model regarding the confidentiality, the integrity, and the availability of the overall system of systems. The considered attacker follows the Dolev-Yao attacker model [11] and most, if not all, practical attacks considered are mitigated in the framework of the proposed security solution. We believe that some potential attacks cannot be truly addressed and we provide justification for this, noting that such attacks are extremely hard, if not impossible, to mitigate in practice, in the context of IoT systems of systems. Therefore, we believe that our solution is highly practical and suitable for the next generation of IoT in the emerging era of systems of systems.

Finally, as already mentioned, some parts of our proposed solution can be modified, extended, or improved, as part of future research. These include, but are not limited to, the customized firmware being used, which cannot yet boot a full Linux kernel, and the usage of an efficient scheme for potential error correction. Additionally, as our proof-of-concept implementation is, at least partially, based on a master-slave device layout and network topology, future research could discuss the applicability of our scheme to even more distributed device architectures and network topologies. Moreover, the proposed security architecture framework can be optimised for particular use cases, such as 5G networks, in an extension of [3], or applied to emerging applications, such as industrial IoT and cyber-physical systems of systems in the evolving context of Industry 4.0, which slowly seems to lead toward the emerging concept of Industry 5.0. Furthermore, the applicability of the proposed solution to use cases that inherently impose time and/or safety restraints, or other requirements, needs to be studied in a more comprehensive manner. It should also be noted that, in the context of IoT 2.0, the proposed security architecture can also be extended to take advantage of novel developments in the fields of artificial intelligence, machine learning, and the blockchain technology.

ACKNOWLEDGMENTS

We would like to thank Oskar Neumann, Jan Philipp Dahms, Mark Rothermel, and Tobias Stensbeck, who contributed to this work in the context of the project topic “Implementation of an IoT/Cyber-Physical System-of-Systems Network” in the framework of the practical/internship course “Bachelor-Praktikum” of the Computer Science Department of the Technical University of Darmstadt in the Winter Semester 2018–2019.

Finally, we would also like to thank the anonymous reviewers of this work for their thoughtful comments and efforts toward improving our original manuscript submission.

REFERENCES

- [1] Ugnius Aliubavicius, Julia Obermaier, Walid Fourati, Diamantis Manolis, Iakovos T. Michailidis, Christina Diakaki, Elias B. Kosmatopoulos, and Michael Krause. 2016. Use of system of systems and decentralized optimization concepts for integrated traffic control via dynamic signalization and embedded speed recommendation. *Transpor. Res. Procedia* 14 (2016), 3416–3425. <https://doi.org/10.1016/j.trpro.2016.05.300> Transport Research Arena TRA2016.
- [2] Muhammad N. Aman, Kee Chaing Chua, and Biplab Sikdar. 2016. Position paper: Physical unclonable functions for IoT security. In *Proceedings of the 2nd ACM International Workshop on IoT Privacy, Trust, and Security (IoTPTS'16)*. ACM, New York, NY, 10–13. <https://doi.org/10.1145/2899007.2899013>

- [3] Nikolaos Athanasios Anagnostopoulos, Saad Ahmad, Tolga Arul, Daniel Steinmetzer, Matthias Hollick, and Stefan Katzenbeisser. 2020. Low-cost security for next-generation IoT networks. *ACM Trans. Internet Technol.* 20, 3 (Sept. 2020), Article 30, 31 pages. <https://doi.org/10.1145/3406280>
- [4] N. A. Anagnostopoulos, T. Arul, Y. Fan, C. Hatzfeld, J. Lotichius, R. Sharma, F. Fernandes, F. Tehranipoor, and S. Katzenbeisser. 2018. Securing IoT devices using robust DRAM PUFs. In *2018 Global Information Infrastructure and Networking Symposium (GIIS'18)*. IEEE. <https://doi.org/10.1109/GIIS.2018.8635789>
- [5] Shuai Chen, Bing Li, and Yuan Cao. 2019. Intrinsic physical unclonable function (PUF) sensors in commodity devices. *Sensors* 19, 11 (May 2019), 2428. <https://doi.org/10.3390/s19112428>
- [6] Shuai Chen, Bing Li, and Cenjun Zhou. 2018. FPGA implementation of SRAM PUFs based cryptographically secure pseudo-random number generator. *Microprocess. Microsyst.* 59 (2018), 57–68. <https://doi.org/10.1016/j.micpro.2018.02.001>
- [7] Shuai Chen, Wenjie Xiong, Yehan Xu, Bing Li, and Jakub Szefer. 2019. Thermal covert channels leveraging package-on-package DRAM. In *Proceedings of the International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom'19)*. <https://doi.org/10.1109/TrustCom/BigDataSE.2019.00050>
- [8] F. Coallier. 2018. A system of systems engineering perspective on IoT trustworthiness. In *2018 13th Annual Conference on System of Systems Engineering (SoSE'18)*. 89–91. <https://doi.org/10.1109/SYSE.2018.8428757>
- [9] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam Smith. 2008. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. *SIAM J. Comput.* 38, 1 (2008), 97–139. <https://doi.org/10.1137/060651380>
- [10] Yevgeniy Dodis, Leonid Reyzin, and Adam Smith. 2004. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In *Advances in cryptology—Eurocrypt 2004*. Springer, 523–540. https://doi.org/10.1007/978-3-540-24676-3_31
- [11] D. Dolev and A. Yao. 1983. On the security of public key protocols. *IEEE Trans. Inf. Theory* 29, 2 (1983), 198–208.
- [12] M. A. Feki, F. Kawsar, M. Boussard, and L. Trappeniers. 2013. The Internet of Things: The next technological revolution. *Computer* 46, 2 (Feb. 2013), 24–25. <https://doi.org/10.1109/MC.2013.63>
- [13] A. P. Fournaris, A. Lalos, P. Kapsalas, and C. Koulamas. 2020. Decentralized, secure and cognitive architecture for automotive cyberphysical system of systems. In *2020 9th Mediterranean Conference on Embedded Computing (MECO'20)*. 1–5. <https://doi.org/10.1109/MECO49872.2020.9134336>
- [14] Jorge Guajardo, Sandeep S. Kumar, Geert-Jan Schrijen, and Pim Tuyls. 2007. FPGA intrinsic PUFs and their use for IP protection. In *Proceedings of the 9th International Workshop on Cryptographic Hardware and Embedded Systems (CHES'07)*. Springer-Verlag, Berlin, 63–80. https://doi.org/10.1007/978-3-540-74735-2_5
- [15] B. Halak, M. Zwolinski, and M. S. Mispan. 2016. Overview of PUF-based hardware security solutions for the Internet of Things. In *2016 IEEE 59th International Midwest Symposium on Circuits and Systems (MWSCAS'16)*. IEEE, 1–4. <https://doi.org/10.1109/MWSCAS.2016.7870046>
- [16] Helena Handschuh. 2012. Hardware-anchored security based on SRAM PUFs, Part 2. *IEEE Secur. Privacy* 10, 4 (July 2012), 80–81. <https://doi.org/10.1109/MSP.2012.98>
- [17] Helena Handschuh, Geert-Jan Schrijen, and Pim Tuyls. 2010. Hardware intrinsic security from physically unclonable functions. In *Towards Hardware-Intrinsic Security*. Springer, 39–53. https://doi.org/10.1007/978-3-642-14452-3_2
- [18] C. Herder, M. D. Yu, F. Koushanfar, and S. Devadas. 2014. Physical unclonable functions and applications: A tutorial. *Proc. IEEE* 102, 8 (Aug. 2014), 1126–1141. <https://doi.org/10.1109/JPROC.2014.2320516>
- [19] G. Hospodar, R. Maes, and I. Verbauwhede. 2012. Machine learning attacks on 65nm Arbiter PUFs: Accurate modeling poses strict bounds on usability. In *2012 IEEE International Workshop on Information Forensics and Security (WIFS'12)*. 37–42. <https://doi.org/10.1109/WIFS.2012.6412622>
- [20] Mo Jamshidi (Ed.). 2008. *System of Systems Engineering*. John Wiley & Sons, Inc. <https://doi.org/10.1002/9780470403501>
- [21] D. Ki-Aries. 2018. Assessing security risk and requirements for systems of systems. In *2018 IEEE 26th International Requirements Engineering Conference (RE'18)*. 454–459. <https://doi.org/10.1109/RE.2018.00061>
- [22] Florian Kohnhäuser, André Schaller, and Stefan Katzenbeisser. 2015. PUF-based software protection for low-end embedded devices. In *Trust and Trustworthy Computing*, Mauro Conti, Matthias Schunter, and Ioannis Askoxylakis (Eds.). Springer International Publishing, Cham, 3–21. https://doi.org/10.1007/978-3-319-22846-4_1
- [23] C. Lipps, M. Strufe, S. B. Mallikarjun, and H. D. Schotten. 2019. Physical layer security for IIoT and CPPS: A cellular-network security approach. In *Mobile Communication—Technologies and Applications; 24. ITG-Symposium*. 1–5.
- [24] R. Maes and I. Verbauwhede. 2010. Physically unclonable functions: A study on the state of the art and future research directions. In *Towards Hardware-Intrinsic Security*. Springer, 3–37. https://doi.org/10.1007/978-3-642-14452-3_1
- [25] D. Mukhopadhyay. 2016. PUFs as promising tools for security in Internet of Things. *IEEE Design Test* 33, 3 (June 2016), 103–115. <https://doi.org/10.1109/MDAT.2016.2544845>
- [26] S. Naik and V. Maral. 2017. Cyber security—IoT. In *2017 2nd IEEE International Conference on Recent Trends in Electronics, Information Communication Technology (RTEICT'17)*. 764–767. <https://doi.org/10.1109/RTEICT.2017.8256700>

- [27] M. A. Olivero, A. Bertolino, F. J. Dominguez-Mayo, M. J. Escalona, and I. Matteucci. 2019. Security assessment of systems of systems. In *2019 IEEE/ACM 7th International Workshop on Software Engineering for Systems-of-Systems (SESoS'19) and 13th Workshop on Distributed Software Development, Software Ecosystems and Systems-of-Systems (WDES'19)*, 62–65. <https://doi.org/10.1109/SESoS/WDES.2019.00017>
- [28] D. Püllen, N. A. Anagnostopoulos, T. Arul, and S. Katzenbeisser. 2019. Using implicit certification to efficiently establish authenticated group keys for in-vehicle networks. In *2019 IEEE Vehicular Networking Conference (VNC'19)*, 1–8. <https://doi.org/10.1109/VNC48660.2019.9062785>
- [29] Ulrich Rührmair, Frank Sehnke, Jan Sölter, Gideon Dror, Srinivas Devadas, and Jürgen Schmidhuber. 2010. Modeling attacks on physical unclonable functions. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS'10)*. ACM, New York, NY, 237–249. <https://doi.org/10.1145/1866307.1866335>
- [30] U. Rührmair, J. Sölter, F. Sehnke, X. Xu, A. Mahmoud, V. Stoyanova, G. Dror, J. Schmidhuber, W. Burleson, and S. Devadas. 2013. PUF modeling attacks on simulated and silicon data. *IEEE Trans. Inf. Forensics Security* 8, 11 (Nov. 2013), 1876–1891. <https://doi.org/10.1109/TIFS.2013.2279798>
- [31] A. Schaller, W. Xiong, N. A. Anagnostopoulos, M. U. Saleem, S. Gabmeyer, B. Škorić, S. Katzenbeisser, and J. Szefer. 2019. Decay-based DRAM PUFs in commodity devices. *IEEE Trans. Dependable Secure Comput.* 16, 3 (2019), 462–475. <https://doi.org/10.1109/TDSC.2018.2822298>
- [32] Steffen Schulz, André Schaller, Florian Kohnhäuser, and Stefan Katzenbeisser. 2017. Boot Attestation: Secure remote reporting with off-the-shelf IoT sensors. In *Computer Security—ESORICS 2017*, Simon N. Foley, Dieter Gollmann, and Einar Snekkenes (Eds.). Springer International Publishing, Cham, 437–455. https://doi.org/10.1007/978-3-319-66399-9_24
- [33] M. Seaborn and T. Dullien. 2015. Exploiting the DRAM Rowhammer bug to gain kernel privileges. Black Hat USA, Mandalay Bay, Las Vegas, NV.
- [34] Argiris Sideris, Dimitris Tsiktsiris, Dimitris Ziouzos, and Minas Dasygenis. 2019. *Smart Grid Hardware Security*. Springer International Publishing, Cham, 85–113. https://doi.org/10.1007/978-3-030-03640-9_5
- [35] Anthony Van Herrewege, Stefan Katzenbeisser, Roel Maes, Roel Peeters, Ahmad-Reza Sadeghi, Ingrid Verbauwhede, and Christian Wachsmann. 2012. Reverse fuzzy extractors: Enabling lightweight mutual authentication for PUF-enabled RFIDs. In *Financial Cryptography and Data Security*, Angelos D. Keromytis (Ed.). Springer, Berlin, 374–389. https://doi.org/10.1007/978-3-642-32946-3_27
- [36] J. R. Wallrabenstein. 2016. Practical and secure IoT device authentication using physical unclonable functions. In *2016 IEEE 4th International Conference on Future Internet of Things and Cloud (FiCloud'16)*. IEEE, 99–106. <https://doi.org/10.1109/FiCloud.2016.22>
- [37] Xiaolin Xu and Wayne Burleson. 2014. Hybrid side-channel/machine-learning attacks on PUFs: A new threat? In *Proceedings of the Conference on Design, Automation & Test in Europe (DATE'14)*. European Design and Automation Association, 3001 Leuven, Belgium, Belgium, Article 349, 6 pages. <http://dl.acm.org/citation.cfm?id=2616606.2617100>
- [38] W. Yan, C. Jin, F. Tehranipoor, and J. A. Chandy. 2017. Phase calibrated ring oscillator PUF design and implementation on FPGAs. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL'17)*. IEEE, 1–8. <https://doi.org/10.23919/FPL.2017.8056859>
- [39] A. Šurković, D. Hanić, E. Lisova, A. Čaušević, D. Wenslandt, and C. Falk. 2018. Towards attack models in autonomous systems of systems. In *2018 13th Annual Conference on System of Systems Engineering (SoSE'18)*, 583–585. <https://doi.org/10.1109/SYSE.2018.8428701>

Received June 2020; revised February 2021; accepted March 2021